Energy-Aware Heterogeneous Deployment of Learned Image Codecs

George Paul
School of Computing
Newcastle University
Newcastle upon Tyne, UK
g.paul2@newcastle.ac.uk

Robert Nicol
STMicroelectronics
Edinburgh, UK
robert.nicol@st.com

Boguslaw Obara
School of Computing
Newcastle University
Newcastle upon Tyne, UK
boguslaw.obara@newcastle.ac.uk

Deepayan Bhowmik
School of Computing
Newcastle University
Newcastle upon Tyne, UK
deepayan.bhowmik@newcastle.ac.uk

***Abstract*—Learned image compression (LIC) achieves state-of-the-art compression performance, yet practical deployments often overlook latency and energy—critical constraints for on-device pipelines and bandwidth-limited applications such as drones and satellite imaging. We address this gap with a codec-aware heterogeneous deployment partitioned at the codec's natural bottleneck—the *quantised latent*—and mapped across devices: large analysis/synthesis transforms to the GPU (high parallel throughput), the compact, pipelined hyperprior to the FPGA (low-power dataflow), and entropy coding to the CPU. The system remains bit-exact with the reference codec and integrates directly with pretrained CompressAI models via exported ONNX subgraphs. Across two pretrained codecs and 1,000 test images, the heterogeneous mapping improves *encoder* runtime by about 10% and reduces encoder energy by 80% on average relative to GPU-only execution. For the *decoder*, runtime remains within 1% of the GPU while energy decreases by approximately 78%. Overall, it achieves an energy–delay product about 80% lower than GPU-only and 29% lower than FPGA-only, providing a superior latency–energy trade-off for LIC deployment.**

***Index Terms*—Learned Image Compression, Heterogeneous Computing, Latency, Energy Efficiency**

I. Introduction

Image compression underpins vision workloads on phones, cameras, and embedded systems. Standards such as JPEG [1]—followed by intra-frame modes of HEVC [2] and AV1 [3]—have defined the baseline. Learned image compression (LIC) has advanced performance through neural encoders, decoders, and hyperpriors [4]–[7]; toolkits like CompressAI [8] and TensorFlow Compression [9] support reproducible evaluation, and *JPEG AI* [10] signals progress toward standardising learned codecs.

While LIC typically optimises rate–distortion, deployed systems must also meet *runtime* and *energy* budgets [11]–[13]. Single-accelerator choices impose trade-offs: GPUs offer high throughput but draw more power [14]; FPGAs improve energy efficiency but require careful mapping [15]; ASICs trade flexibility for peak efficiency [16]. Modern platforms integrate CPUs, GPUs, and FPGAs [17], yet LIC rarely exploits this heterogeneity. To our knowledge, this is the first energy-aware deployment of learned image codecs across heterogeneous hardware that jointly improves latency and energy.

We propose a codec-aware partition at the encoder's discrete latent code $\hat{\mathbf{y}}$, ensuring that only a compact payload traverses PCIe while each transform runs on its most suitable device. This yields GPU-class latency and near-FPGA energy while preserving the reference bitstream and reconstructions. Our contributions are:

- **Codec-aware heterogeneous partitioning:** a split at the quantised latent that minimises inter-device data movement while preserving compression performance.
- **Software–hardware workflow:** a Python, ONNX [18], partitioning, and runtime pipeline that schedules subgraphs and intermediate transfers, yielding predictable CPU/GPU/FPGA mapping.
- **Plug-and-play heterogeneous deployment:** execution across CPU, GPU, and FPGA preserves bitstream and reconstruction fidelity, enabling drop-in use of pretrained LIC models.
- **Demonstrated latency–energy gains:** mapping analysis/synthesis to the GPU, the hyperprior to the FPGA, and entropy coding to the CPU achieves GPU-class latency at substantially lower energy.

II. Related Work

Learned image compression. LIC replaces handcrafted transforms with neural networks: an analysis transform maps an image to a latent space that is quantised and entropy-coded by a learned probability model; a synthesis transform reconstructs pixels. Hyperpriors introduce side information and spatial context to improve compression [19]. Variable-rate, content-adaptive algorithms allocate bits where they are perceptually most valuable [20]. These advances increase compute costs, particularly in decoder and entropy paths, so deployment must consider *runtime* and *energy*, not only rate–distortion.

Hardware acceleration of learned codecs. FPGA and ASIC implementations accelerate LIC codecs and reduce power consumption [21]–[23]. FPGA pipelines exploit streaming dataflow and fixed-point arithmetic with on-chip buffers but require extensive manual tuning and sacrifice portability. ASICs achieve higher efficiency via custom arithmetic and tightly coupled memory systems, yet lose flexibility as codec architectures evolve.

Heterogeneous and cross-device deployment. Heterogeneous inference across CPUs, GPUs, and FPGAs explores

partitioning and scheduling to balance compute and communication [24]–[29]. Fine-grained splits increase PCIe traffic and coordination; coarse mapping underutilises hardware. Existing frameworks primarily target CNN inference, not codec-specific operators (probability modelling, entropy coding) that demand strict determinism in rounding, layout, and symbol exchange, narrowing viable partitions.

Our approach. We propose a codec-aware heterogeneous deployment flow that aligns the hardware boundary at the quantised latent. It preserves bit-exact fidelity, minimises PCIe traffic by exchanging only compact codes, and enables coordinated execution across accelerators. Unlike layer-wise or single-accelerator mapping, our deterministic interface supports practical multi-device deployment while maintaining reference behaviour—offering an energy-efficient path toward standardised learned codecs such as JPEG AI.

III. Methodology

We strive to optimise the latency–energy trade-off in LIC codecs using heterogeneous hardware. We proceed in three steps: (i) profile encoder/decoder subgraphs across devices, (ii) choose a codec-aware partition, and (iii) implement a deterministic heterogeneous deployment flow (Fig. 1).

A. Profiling

We benchmark per-stage runtime for two representative LIC codecs—`bmshj2018-hyperprior` [5] and `mbt2018-mean` [6]—in CompressAI [8]. Subgraphs (Fig. 2) are S1a/S1b (encoder and quantiser), S2a/S4a (hyperprior), and S3a/S3b (decoder). Table I reports per-stage means at 512×512 resolution over $N=1000$ images. Bandwidth-bound transforms favour the GPU, whereas the compact, reuse-intensive hyperprior favours the FPGA. Arithmetic encoding and decoding for both $\hat{\mathbf{y}}$ and $\hat{\mathbf{z}}$ run on the CPU. Only the discrete latent is exchanged—no extra GPU–FPGA PCIe traffic for CPU AE/AD. Shapes, convolution algorithms, and deterministic settings are fixed across runs. These measurements motivate the mapping in Table II.

B. Codec Partitioning

Given an input image $\mathbf{x}\in\mathbb{R}^{1\times3\times H\times W}$, the encoder analysis transform g_a produces features $\mathbf{y}=g_a(\mathbf{x})$ and a discrete code

TABLE I
PER-STAGE RUNTIME (MS) AT 512×512 RESOLUTION ACROSS CPU, GPU (FP16), AND FPGA (U50, INT8 WITH INT32 ACCUMULATORS). MEANS OVER $N=1000$. *Bold* = FASTEST.

Model	Stage	CPU	GPU	FPGA
`bmshj2018-hyperprior`	$g_a{+}Q$ (enc)	3.8	**0.95**	1.9
	h_a (enc)	0.62	0.18	**0.11**
	h_s (dec)	0.61	0.18	**0.10**
	g_s (dec)	4.2	**0.88**	2.0
`mbt2018-mean`	$g_a{+}Q$ (enc)	3.4	**0.82**	1.7
	h_a (enc)	0.49	0.14	**0.09**
	h_s (dec)	0.48	0.14	**0.09**
	g_s (dec)	3.8	**0.79**	1.8

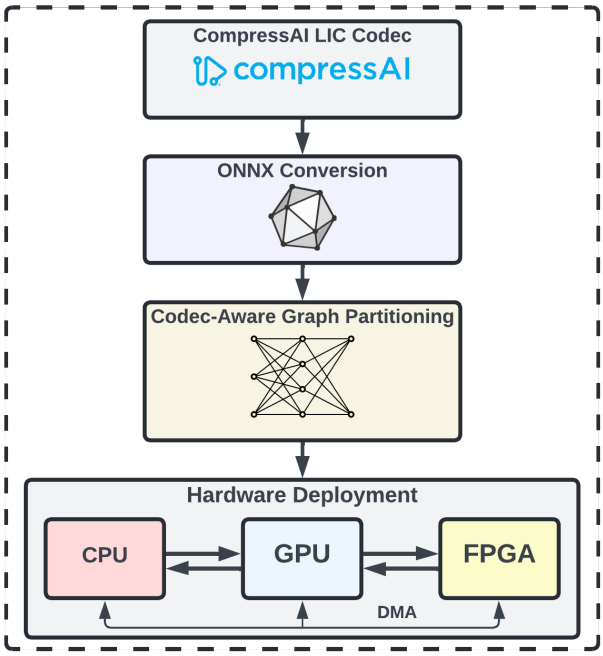

Fig. 1. End-to-end heterogeneous deployment: CompressAI → ONNX subgraphs → partition at $\hat{\mathbf{y}}$ → execution on GPU (g_a, g_s), FPGA (h_a, h_s), and CPU (AE/AD).

$\hat{\mathbf{y}}=Q(\mathbf{y})\in\mathbb{Z}^{1\times C\times(H/D)\times(W/D)}$, where Q is uniform scalar quantisation, C is the number of latent channels, and D is the encoder downsampling factor. A hyper-analysis/synthesis path generates side information as

$$\hat{\mathbf{z}} = Q\big(h_a(\hat{\mathbf{y}})\big), \qquad \psi = h_s(\hat{\mathbf{z}}), \tag{1}$$

where h_a and h_s denote the hyper-analysis and hyper-synthesis transforms (hyperprior), and ψ denotes the decoded hyperprior parameters that condition the entropy model $p(\hat{\mathbf{y}} \mid \psi)$. The bitstream $\mathcal{B}$ is formed by entropy coding $\hat{\mathbf{y}}$ conditioned on ψ. The decoder reconstructs $\hat{\mathbf{x}}=g_s(\hat{\mathbf{y}})$. We place the device boundary at $\hat{\mathbf{y}}$ to minimise PCIe traffic while keeping quantisation and probability modelling on their native sides. As shown in Table I, g_a/g_s are bandwidth-dominated and favour the GPU, while h_a/h_s are reuse-heavy with compact working sets that fit in FPGA on-chip SRAM, favouring streaming dataflow. Accordingly, Table II maps g_a/g_s to the GPU, h_a/h_s to the FPGA, and AE/AD to the CPU. The heterogeneous partition is illustrated in Fig. 2.

C. Heterogeneous Deployment

We define a latent-centric interface at $\hat{\mathbf{y}}$: fixed layout, static shapes and strides, defined endianness, and ties-to-even rounding, enabling bit-exact PACK/UNPACK across devices. FP16 is used for GPU transforms (g_a, g_s) and INT8 with `int32` accumulators for the FPGA hyperprior (h_a, h_s). To stabilise INT8 quantisation in h_a/h_s, GDN (Generalised Divisive Normalisation) [30] layers are replaced with ReLU to

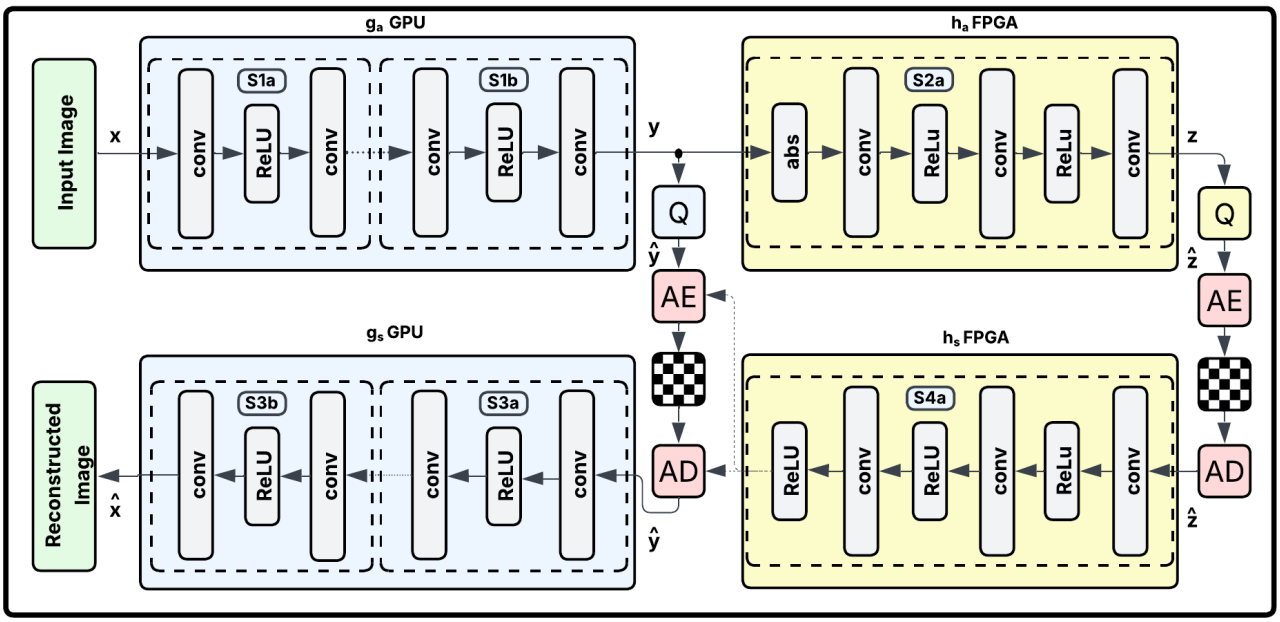

Fig. 2. Heterogeneous codec partitioning overview. The device boundary is placed at the quantised latent $\hat{\mathbf{y}}$, so that only packed quantised latents cross PCIe once per direction with DMA overlap. The analysis and synthesis transforms (g_a, g_s) execute on the GPU, the hyperprior transforms (h_a/h_s) execute on the FPGA, and arithmetic encoding/decoding for both $\hat{\mathbf{y}}$ and $\hat{\mathbf{z}}$ executes on the CPU. Subgraphs used for profiling and deployment are explicitly defined: S1a (early analysis transform layers of g_a), S1b (remaining g_a layers and quantiser Q producing $\hat{\mathbf{y}}$), S2a/S4a (hyperprior h_a/h_s), and S3a (early synthesis transform layers of g_s) and S3b (remaining g_s reconstruction layers).

avoid division-heavy nonlinearities, while preserving the codec structure and entropy interface used for evaluation.

We implement a software–hardware workflow for heterogeneous deployment, comprising Python model description, ONNX export, optimisation through subgraph partitioning, and generation of the final bitstream. The runtime schedules GPU, FPGA, and CPU subgraphs with double buffering and overlapped PCIe transfers, while intermediate latents are moved through direct-memory-access queues to hide interconnect latency [31]. Fig. 1 summarises the flow: a pretrained CompressAI model is exported to ONNX subgraphs (S1a/S1b, S2a/S4a, S3a/S3b); the device cut is aligned to $\hat{\mathbf{y}}$; and execution maps g_a/g_s to GPU, h_a/h_s to FPGA, and AE/AD to CPU. A single packed-$\hat{\mathbf{y}}$ transfer per direction overlaps with compute via DMA; the bitstream is formed and consumed on the CPU in host memory. Table II summarises subgraphs, placement, and rationale.

For latency prediction, let $T_{\text{enc}}, T_{\text{dec}}$ be total encode/decode latency; T_{S}, subgraph latency; T_{ent} CPU entropy time; and T_{dma} PCIe time for packed $\hat{\mathbf{y}}$ for steady-state streaming execution:

$$T_{\text{enc}} \approx \max\{T_{\text{S1a}} + T_{\text{S1b}},\ T_{\text{dma}} + T_{\text{S2a}} + T_{\text{ent,enc}}\}, \quad (2)$$

$$T_{\text{dec}} \approx \max\{T_{\text{S3a}} + T_{\text{S3b}},\ T_{\text{ent,dec}} + T_{\text{dma}} + T_{\text{S4a}}\}. \quad (3)$$

Since only a compact $\hat{\mathbf{y}}$ crosses PCIe, T_{dma} is small, consistent with Table I.

TABLE II
SUBGRAPHS AROUND $\hat{\mathbf{y}}$: PLACEMENT AND RATIONALE.

Subgraph	Role	HW	Justification
S1a	Early encoder (down/conv)	GPU (FP16)	Large feature maps; high memory bandwidth and tensor parallelism.
S1b	Late encoder + quantise	GPU (FP16)	Cut emits $\hat{\mathbf{y}}$; single compact DMA; avoids extra sync.
S2a	Hyper-analysis	FPGA (INT8)	Reuse-dense convs; small working set fits on-chip; efficient streaming.
S4a	Hyper-synthesis	FPGA (INT8)	Produces ψ locally; symmetry with S2a; keeps side-info off PCIe.
S3a	Early decoder (up/conv)	GPU (FP16)	Upsampling and wide convs benefit from GPU bandwidth and compute.
S3b	Late decoder (reconstruct)	GPU (FP16)	Final image on GPU; avoids returning to FPGA/CPU mid-flow.

IV. EXPERIMENTAL RESULTS

Experiments use an AMD Ryzen 9 5900X (CPU), NVIDIA RTX A2000 (GPU), and Xilinx Alveo U50 (FPGA). GPU

TABLE III

MEAN PER-FRAME END-TO-END ENCODER AND DECODER PATH RUNTIME, ENERGY, AND ENERGY–DELAY PRODUCT (EDP) FOR EACH CODEC ON 512×512 RESOLUTION TEST IMAGES; N=1000 INFERENCES. EDP IN µJ·S. EVALUATED AT 1.00 BPP (QUALITY q=4). BEST (LOWEST EDP) IN **BOLD**; SECOND-BEST UNDERLINED.

Model / Path	CPU			GPU			FPGA			Heterogeneous		
	Runtime (ms)	Energy (J)	EDP (µJ·s)	Runtime (ms)	Energy (J)	EDP (µJ·s)	Runtime (ms)	Energy (J)	EDP (µJ·s)	Runtime (ms)	Energy (J)	EDP (µJ·s)
`bmshj2018-hyperprior-encoder`	1.57	0.154	241.8	0.48	0.092	44.2	0.73	0.015	11.0	0.43	0.018	**7.7**
`bmshj2018-hyperprior-decoder`	1.63	0.160	260.8	0.47	0.090	42.3	0.75	0.016	12.0	0.45	0.019	**8.6**
`mbt2018-mean-encoder`	1.34	0.129	172.9	0.42	0.079	33.2	0.63	0.013	8.2	0.38	0.016	**6.1**
`mbt2018-mean-decoder`	1.46	0.140	204.4	0.40	0.078	31.2	0.69	0.015	10.4	0.41	0.017	**7.0**

executes CUDA kernels; FPGA uses Vitis with GPUDirect RDMA; CPU runs entropy coding. Evaluated codecs are `bmshj2018-hyperprior` and `mbt2018-mean`. The dataset is MS-COCO 2017 val [32], centre-cropped and resized to 512×512 pixels; N=1000 images are processed in fixed order. All experiments use 1.00 bpp and quality q=4, corresponding to medium–high visual quality typical of LIC benchmarks. The deployment flow uses fixed scheduling, static DMA queues, and bounded PCIe transfers to reduce latency variance across runs.

Measurements: Runtime is measured with `std::chrono` (CPU), CUDA events (GPU), and Vitis counters (FPGA). Power is sampled via RAPL [33], `nvidia-smi` [34], and `xbutil` [35], then integrated per frame with idle power removed and summed across devices.

Metrics: For frames $i = 1, \ldots, N$,

$$\bar{t} = \frac{1}{N}\sum_i t_i \text{ (ms)}, \qquad \bar{E} = \frac{1}{N}\sum_i E_i \text{ (J)}, \tag{4}$$

with 95% bootstrap confidence intervals (10^4 resamples).

Runtime and Energy: Fig. 3 shows mean encoder/decoder runtime (ms) and energy (J) at 512×512 resolution. CPU is slowest and most energy-demanding; GPU is fastest but still power-hungry; FPGA is most efficient. The heterogeneous mapping attains near-GPU runtime with near-FPGA energy. Relative to GPU-only (Table III), runtime drops by 7.4%/3.7% and energy by 79.7%/79.0% (enc/dec), averaging 5.6% faster runtime and 79% lower energy.

Energy–Delay Product (EDP): EDP combines per-frame energy and latency into a single objective for fair comparison.

$$\text{EDP} = \bar{E}\cdot\frac{\bar{t}}{1000}\ (\mu\text{J}\cdot\text{s/frame}), \tag{5}$$

Fig. 4 and Table III show the heterogeneous configuration achieves the lowest EDP—81% and 80% below GPU-only and roughly 29% below FPGA-only.

Across both codecs and all hardware configurations, the heterogeneous system minimises the latency-energy cost.

V. CONCLUSION

We present a codec-aware heterogeneous LIC deployment that partitions at the quantised latent and couples a software–hardware flow with a scheduling runtime to preserve rate–distortion performance and yield predictable per-accelerator CPU/GPU/FPGA mappings on our experimental setup. The GPU runs the analysis/synthesis transforms, the FPGA the hyperprior, and the CPU entropy coding. On images of resolution 512×512, the *encoder* is ≈ 10% faster with ≈ 80% lower energy than GPU-only; the *decoder* matches GPU-only latency within 1% with ≈ 78% lower energy. Overall, the energy–delay product falls by ≈ 80% vs. GPU-only and 29% vs. FPGA-only, without affecting rate–distortion. In future work we will sweep bit-rates and quality levels to characterise scalability across targets, apply computational-graph optimisations, and deploy LIC codecs on emerging edge accelerators such as NPUs [36].

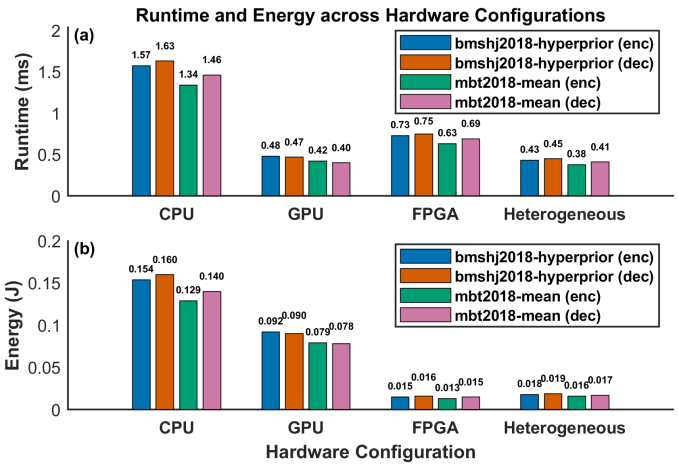

Fig. 3. Mean encoder and decoder runtime (ms) and energy (J) per device; lower is better. Heterogeneous mapping achieves near-GPU speed with near-FPGA energy.

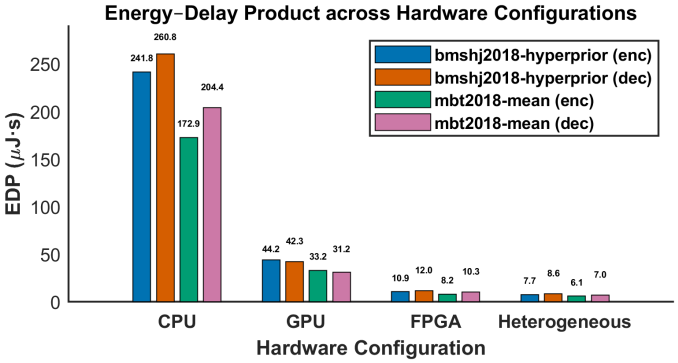

Fig. 4. Encoder and decoder Energy–Delay Product (EDP) (µJ·s); lower is better. Heterogeneous mapping achieves minimum EDP across devices.

REFERENCES

[1] G. K. Wallace, "The JPEG still picture compression standard," *Commun. ACM*, vol. 34, no. 4, pp. 30–44, Apr. 1991. [Online]. Available: https://doi.org/10.1145/103085.103089

[2] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the high efficiency video coding (HEVC) standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 22, no. 12, pp. 1649–1668, 2012.

[3] J. Han, B. Li, D. Mukherjee, C.-H. Chiang, A. Grange, C. Chen, H. Su, S. Parker, S. Deng, U. Joshi, Y. Chen, Y. Wang, P. Wilkins, Y. Xu, and J. Bankoski, "A technical overview of AV1," *Proceedings of the IEEE*, vol. PP, pp. 1–28, 02 2021.

[4] J. Ballé, V. Laparra, and E. P. Simoncelli, "End-to-end optimized image compression," 2017. [Online]. Available: https://arxiv.org/abs/1611.01704

[5] J. Ballé, D. Minnen, S. Singh, S. J. Hwang, and N. Johnston, "Variational image compression with a scale hyperprior," 2018. [Online]. Available: https://arxiv.org/abs/1802.01436

[6] D. Minnen, J. Ballé, and G. Toderici, "Joint autoregressive and hierarchical priors for learned image compression," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, ser. NIPS'18. Red Hook, NY, USA: Curran Associates Inc., 2018, pp. 10 794–10 803.

[7] Z. Cheng, H. Sun, M. Takeuchi, and J. Katto, "Learned image compression with discretized gaussian mixture likelihoods and attention modules," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 7936–7945.

[8] J. Bégaint, F. Racapé, S. Feltman, and A. Pushparaja, "CompressAI: A PyTorch library and evaluation platform for end-to-end compression research," *arXiv preprint arXiv:2011.03029*, 2020.

[9] J. Ballé, S. J. Hwang, and E. Agustsson, "TensorFlow Compression: Learned data compression," 2024. [Online]. Available: http://github.com/tensorflow/compression

[10] J. a. Ascenso, E. Alshina, T. Ebrahimi, and D. Grois, "The JPEG AI standard: Providing efficient human and machine visual data consumption," *IEEE MultiMedia*, vol. 30, no. 1, pp. 100–111, Jan. 2023. [Online]. Available: https://doi.org/10.1109/MMUL.2023.3245919

[11] D. He, Z. Yang, W. Peng, R. Ma, H. Qin, and Y. Wang, "ELIC: Efficient learned image compression with unevenly grouped space-channel contextual adaptive coding," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 5708–5717.

[12] S. W. Hasinoff, D. Sharlet, R. Geiss, A. Adams, J. T. Barron, F. Kainz, J. Chen, and M. Levoy, "Burst photography for high dynamic range and low-light imaging on mobile cameras," *ACM Trans. Graph.*, vol. 35, no. 6, Dec. 2016. [Online]. Available: https://doi.org/10.1145/2980179.2980254

[13] X. Wang, Z. Tang, J. Guo, T. Meng, C. Wang, T. Wang, and W. Jia, "Empowering edge intelligence: A comprehensive survey on on-device AI models," *ACM Comput. Surv.*, vol. 57, no. 9, Apr. 2025. [Online]. Available: https://doi.org/10.1145/3724420

[14] K. Yoshida, R. Sageyama, S. Miwa, H. Yamaki, and H. Honda, "Analyzing performance and power-efficiency variations among NVIDIA GPUs," in *Proceedings of the 51st International Conference on Parallel Processing*, ser. ICPP '22. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3545008.3545084

[15] H. Sun, Q. Yi, and M. Fujita, "FPGA codec system of learned image compression with algorithm-architecture co-optimization," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 14, no. 2, pp. 334–347, 2024.

[16] T. Mendez, T. Parupudi, V. Kedlaya K, and S. G. Nayak, "Development of power-delay product optimized ASIC-based computational unit for medical image compression," *Technologies*, vol. 12, no. 8, p. 121, 2024.

[17] C. Bobda, J. M. Mbongue, P. Chow, M. Ewais, N. Tarafdar, J. C. Vega, K. Eguro, D. Koch, S. Handagala, M. Leeser, M. Herbordt, H. Shahzad, P. Hofste, B. Ringlein, J. Szefer, A. Sanaullah, and R. Tessier, "The future of FPGA acceleration in datacenters and the cloud," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 15, no. 3, Feb. 2022. [Online]. Available: https://doi.org/10.1145/3506713

[18] ONNX Contributors, "ONNX: Open neural network exchange," https://onnx.ai, 2019.

[19] F. Mentzer, E. Agustsson, M. Tschannen, R. Timofte, and L. Van Gool, "Conditional probability models for deep image compression," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[20] O. Rippel and L. Bourdev, "Real-time adaptive image compression," in *Proceedings of the 34th International Conference on Machine Learning – Volume 70*, ser. ICML'17. JMLR.org, 2017, pp. 2922–2930.

[21] J. Chen, Q. Chen, H. Zhang, W. Chen, W. Luo, and F. Yu, "HENCE: Hardware end-to-end neural conditional entropy encoder for lossless 3d medical image compression," *IEEE Access*, 2024.

[22] H. Sun, J. Wang, S. Liu, S. Kimura, and M. Fujita, *Learned Image Codec on FPGA: Algorithm, Architecture and System Design*. New York, NY, USA: Association for Computing Machinery, 2025, pp. 328–331. [Online]. Available: https://doi.org/10.1145/3658617.3698489

[23] W. Gao, "Implementations for AI-based image and video coding," in *AI-Based Image and Video Coding: Methods, Standards, and Applications*. Springer, 2025, pp. 271–303.

[24] S. I. Venieris, A. Kouris, and C.-S. Bouganis, "Toolflows for mapping convolutional neural networks on FPGAs: A survey and future directions," *ACM Comput. Surv.*, vol. 51, no. 3, Jun. 2018. [Online]. Available: https://doi.org/10.1145/3186332

[25] X. Nie, X. Miao, Z. Yang, and B. Cui, "TSPLIT: Fine-grained GPU memory management for efficient DNN training via tensor splitting," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, 2022, pp. 2615–2628.

[26] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'18. USA: USENIX Association, 2018, pp. 579–594.

[27] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *SIGARCH Comput. Archit. News*, vol. 45, no. 1, pp. 615–629, Apr. 2017. [Online]. Available: https://doi.org/10.1145/3093337.3037698

[28] E. G. Fefey and T. Islam, "Toward efficient deep learning inference: On-node heterogeneous scheduling in edge–cloud infrastructure," in *2024 IEEE Cloud Summit*. IEEE, 2024, pp. 73–78.

[29] C. Li, S. Liu, K. Jiang, M. Yang, Z. Zhang, B. Wang, L. Zhao, C. Chen, and S. Wan, "DNN inference acceleration based on adaptive task partitioning and offloading in embedded VEC," *ACM Trans. Embed. Comput. Syst.*, vol. 24, no. 4, Jun. 2025. [Online]. Available: https://doi.org/10.1145/3725734

[30] J. Ballé, V. Laparra, and E. P. Simoncelli, "Density modeling of images using a generalized normalization transformation," 2016. [Online]. Available: https://arxiv.org/abs/1511.06281

[31] T. Ali, G. Paul, R. Nicol, and D. Bhowmik, "Scheduling algorithms on heterogeneous architecture for efficient vision systems," in *Proceedings of SPIE Applications of Digital Image Processing XLVII*, A. G. Tescher and T. Ebrahimi, Eds., vol. 13137, International Society for Optics and Photonics (SPIE). SPIE, 2024, p. 131370F. [Online]. Available: https://doi.org/10.1117/12.3031278

[32] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *European Conference on Computer Vision (ECCV)*. Springer, 2014, pp. 740–755. [Online]. Available: https://cocodataset.org

[33] H. David, E. Gorbatov, U. R. Hanebutte, R. Khanna, and C. Le, "RAPL: Memory power estimation and capping," in *Proceedings of the 16th ACM/IEEE International Symposium on Low Power Electronics and Design*, ser. ISLPED '10. New York, NY, USA: Association for Computing Machinery, 2010, pp. 189–194. [Online]. Available: https://doi.org/10.1145/1840845.1840883

[34] NVIDIA Corporation, "NVIDIA System Management Interface (nvidia-smi)," https://developer.nvidia.com/nvidia-system-management-interface, 2024.

[35] AMD Xilinx, "Xilinx Board Utility (xbutil)," https://docs.amd.com/r/en-US/ug1301-xrt/xbutil-Utility, 2024.

[36] Hailo Technologies Ltd., "Hailo AI Processors: Deep Learning Accelerators for Edge Devices," https://hailo.ai, 2023.